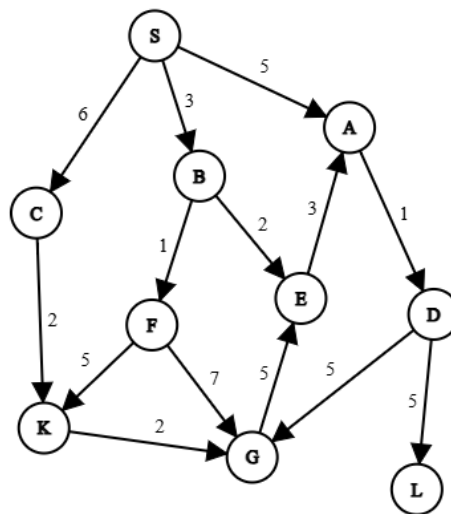




A directed Graph where node S is start, node G is goal node.

Experiment-01: Breadth First Search (BFS)

- Breadth-First Search (BFS) is a fundamental graph traversal algorithm.
- It explores a graph layer by layer, starting from a given source node.
- It visits all direct neighbors of the starting node first, then moves to the neighbors' neighbors, and so on.

Source Code:

```

graph={}
parent={}
with open("graph.txt","r") as file: # storing the graph in nested dictionary
    f=file.readlines()
    for i in f:
        i=i.split(' ')
        if i[0] not in graph:
            graph[i[0]]={}
        graph[i[0]][i[1]]=int(i[2])
        if i[1] not in graph:
            graph[i[1]]={}
file.close()
# print(graph)
start='S'#input("Start node: ")
goal='G'#input("Goal node: ")
path=""
cost=0
queue=[]
queue.append(start)
while len(queue)!=0:
    top=queue[0]

```

```

queue.pop(0)
f=0
for i in graph[top]:
    if i not in parent: # if a child got parent, no parent update
        queue.append(i)
        parent[i[0]]=top
    if top==goal:
        f=1
        break
if f:
    break

while True:
    path+=goal
    next_path=parent[goal]
    cost+=graph[next_path][goal]
    goal=next_path
    if goal==start:
        path+=goal
        break

print(f'path: {path[::-1]}\ncost={cost}')

```

Output:

```

path:SCKG
cost=10
PS F:\RUET CSE LAB\3208-Artificial Intelligence\2. 24-09-2025>

```

Conclusion:

In conclusion, Breadth-First Search (BFS) stands as a fundamental algorithm in computer science, defined by its systematic, layer-by-layer exploration of graph structures. Its reliance on a queue data structure ensures that nodes are processed in the exact order of their distance from the source. This characteristic makes BFS the optimal choice for problems where the goal is to find the shortest path or the minimum number of steps in an unweighted environment. Some applications are Shortest Path Finding, Social Network Analysis, Web Crawling, Broadcasting.

Experiment-02: Depth First Search (DFS)

- Depth-First Search (DFS) is a graph traversal algorithm that prioritizes exploring as "deep" as possible into the graph before backtracking.
- Unlike BFS, which explores neighbors' layer by layer, DFS picks one path and follows it until it hits a dead end, then backtracks to the most recent node with unexplored options.

Source Code:

```
graph={}
parent={}
with open("graph.txt","r") as file: # storing the graph in nested dictionary
    f=file.readlines()
    for i in f:
        i=i.split(' ')
        if i[0] not in graph:
            graph[i[0]]={}
        graph[i[0]][i[1]]=int(i[2])
        if i[1] not in graph:
            graph[i[1]]={}
    file.close()
# print(graph)
start='S'#input("Start node: ")
goal='G'#input("Goal node: ")
path=""
cost=0

queue=[]
queue.append(start)

while len(queue)!=0:
    top=queue[0]
    queue.pop(0)
    f=0
    for i in graph[top]:
        if i not in parent: # if a child got parent, no parent update
            queue.insert(0,i)
            parent[i[0]]=top
        if top==goal:
            f=1
            break
    if f:
        break

while True:
    path+=goal
    next_path=parent[goal]
    cost+=graph[next_path][goal]
    goal=next_path
    if goal==start:
        path+=goal
        break
print(f'path: {path[::-1]}\ncost={cost}')
```

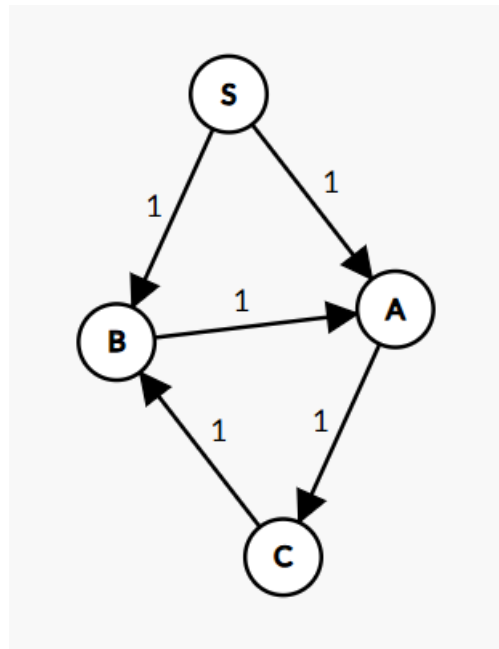
Output:

```
path:SADG
cost=11
PS F:\RUET CSE LAB\3208-Artificial Intelligence\2. 24-09-2025>
```

Conclusion:

In summary, Depth-First Search (DFS) is a recursive, backtracking algorithm that excels at exploring all possibilities along a specific path. Because it dives deep into data structures rather than spreading out, it is particularly memory-efficient for wide graphs and is the natural choice for problems requiring exhaustive searching of a specific branch. Key Applications are Topological Sorting, Cycle Detection, Solving Mazes and Puzzles, Finding Strongly Connected Components, Pathfinding.

Experiment-03: Cycle Detection in Directed Graph



- It is an application of the DFS Algorithm.
- If we start from node and get back the starting node again in our traversal, it will create a cycle or loop.

Source Code:

```
graph={}
parent={}
visited={}

with open("graph2.txt","r") as file: # storing the graph in nested dictionary
    f=file.readlines()
    for i in f:
        i=i.split(' ')
```

```

        if i[0] not in graph:
            graph[i[0]]={}
            graph[i[0]][i[1]]=int(i[2])
        if i[1] not in graph:
            graph[i[1]]={}
file.close()
# print(graph)

start='S'#input("Start node: ")
goal='G'#input("Goal node: ")
path=""
node=[]

queue=[]
queue.append(start)

while len(queue)!=0:
    top=queue[0]
    queue.pop(0)
    visited[top]=1
    for i in graph[top]:
        if i not in visited:
            queue.insert(0,i)
        if i in parent:
            node.append(i) # previous defined parent are stored in node.
            parent[i]=top

c="No Cycle"
if node:
    c="Cycle exists"
    p=""
    f=0
    for i in node:
        p=i
        path=""
        while True:
            path+=p
            try:
                p=parent[p]
            except:
                c="No Cycle"
                path=""
                break
        if p==path[0]:
            path+=p

```

```
        f=1
        break
    if len(path)>len(graph):
        break
    if f:
        break

print(f'{c}\npath: {path[::-1]}')
```

Output:

```
    Cycle exists
    path:ACBA
PS F:\RUET CSE LAB\3208-Artificial Intelligence\2. 24-09-2025>
```

Conclusion:

In conclusion, Cycle Detection in a Directed Graph is a critical validation step in computer science, ensuring that systems involving dependencies do not enter infinite loops or deadlocks.